FINNEWS AI

Financial News Simplifier

Complex Financial News. Made Simple.

PROJECT REPORT

A practical design and implementation guide for an AI-assisted financial news application

Dr. D. Y. Patil College of Engineering and Innovation
Department of Computer Engineering

Submitted By: Mukta Kolhe

Abstract

FinNews AI is a web application concept and implementation blueprint that helps students, retail investors, beginners, and busy professionals understand complex financial, economic, and business reporting. It combines a curated news experience with an AI-assisted simplifier, a structured explanation format, a custom text simplifier, and a financial term explainer. The intended explanation separates reported facts from implications and defines unfamiliar vocabulary in accessible language.

The documented primary implementation is a React 19 client built with Vite and styled with Tailwind CSS, communicating over REST/JSON with a Node.js and Express server. A FastAPI backend is described as an alternate deployment mode behind the same conceptual client boundary. Language model providers are server-side integrations and configurable choices; model names in the original concept are examples, not guaranteed availability or a promise of current provider behavior. The report specifies modules, a proposed API contract, environment setup, security controls, testing strategy, limitations, and future work. Because no source repository was supplied with the project brief, endpoint paths and folder layout below are implementation recommendations to reconcile against the actual code before deployment.

Contents

1. Introduction and problem statement

2. Objectives and scope

3. Requirements and features

4. System architecture

5. Module descriptions

6. Technology stack

7. Project structure

8. API endpoint documentation

9. Environment setup and execution

10. Security considerations

11. Testing approach

12. Limitations and assumptions

13. Future enhancements

14. Conclusion

References

1. Introduction and Problem Statement

1.1 Background

Financial reporting compresses many technical ideas into short articles: inflation measures, interest-rate decisions, company guidance, market movements, and policy changes. Readers may know the headline but not the terms, context, or possible effect on households and portfolios. Search results also mix source reporting, analysis, and opinion, making it difficult for a beginner to separate what happened from why it may matter.

1.2 Problem statement

The project addresses the gap between detailed financial news and the level of explanation needed by non-specialist readers. A useful solution must preserve names, figures, dates, units, and uncertainty while explaining terminology in plain language. It should make clear that an explanation is educational context rather than personalized investment advice.

1.3 Proposed solution

FinNews AI offers categorized business news and turns selected or user-provided text into an accessible, structured explanation. The server mediates calls to external model providers, validates inputs, and can return a clearly labeled fallback when live services are unavailable or rate-limited. The client presents the result in a predictable format that is easier to scan and compare.

2. Objectives and Scope

2.1 Objectives

- Present curated financial news across Markets, Economy, Technology, Banking, Global Markets, Startups, and Cryptocurrency.
- Simplify specialist language without intentionally changing the source meaning or removing important data.
- Organize explanations into a simplified headline, executive summary, objective account, implications, key takeaways, and a jargon buster.
- Let a reader submit an article, press release, or textbook excerpt for an on-demand breakdown.
- Provide quick definitions for financial terms such as basis points, yield curve, core CPI, quantitative easing, and capital expenditure (CapEx).
- Keep provider credentials on the server and design for graceful handling of outages, timeouts, and rate limits.

2.2 Scope

The project scope covers a responsive web client, server API, news-source or feed integration, optional model-provider integration, term lookup, validation, and operational configuration. News availability, provider access, model quality, and response time depend on configured third-party services. The project does not itself guarantee trading data, comprehensive market coverage, financial advice, or investment outcomes.

3. Requirements and Features

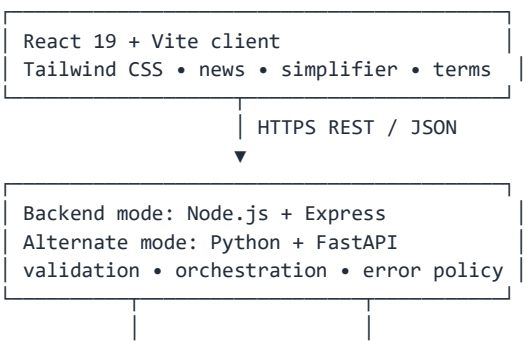
Area	Functional requirement	Expected behavior
News	Browse and filter curated financial stories	Category-based discovery; story metadata and source attribution are retained.
Simplification	Generate a plain-language explanation	Structured sections keep facts, implications, and definitions distinguishable.
Custom input	Submit user-provided text	Input is validated and sent to the configured backend simplification service.
Term lookup	Explain a financial term	Return a concise definition and, where available, a short example.
Resilience	Handle unavailable upstream services	Return a user-safe error or explicitly labeled fallback; do not imply live AI ran when it did not.
Privacy and security	Protect secrets and user content	Provider keys remain server-side; apply validation, limits, and safe logging.

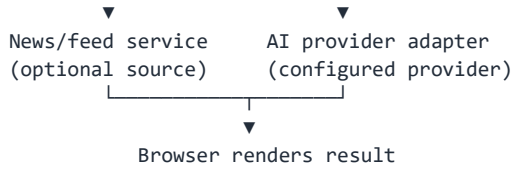
3.1 Structured explanation format

Section	Purpose
Simplified headline	Express the main event in familiar language.
Executive TL;DR	Give a short orientation for readers who need the gist.
What happened?	State source-supported facts, dates, actors, and figures.
Why it matters	Describe plausible relevance to rates, loans, jobs, businesses, or portfolios while labeling uncertainty.
Key takeaways	Summarize the few points a reader should retain.
Jargon buster	Define terms used in the story in plain English.

4. System Architecture

The client and backend communicate through a REST/JSON boundary. The Node/Express version is the primary mode in this report. FastAPI is an alternative backend implementation; it should expose equivalent behavior if the same frontend is to switch between modes. A news provider and an AI model provider are optional external services controlled through server configuration.





4.1 Request flow

1. The client requests stories, a term definition, or a simplification from the backend.
2. The backend validates the request, applies size and rate limits, and selects a configured news or model adapter.
3. The adapter calls the upstream service using server-held credentials and a bounded timeout.
4. The backend normalizes the result to its response contract, records safe operational metadata, and returns JSON.
5. The client displays results, loading states, validation feedback, attribution, and service errors.

4.2 Backend modes

Mode	Description	Configuration note
Primary: Express	Node.js server with Express routes and middleware. Suitable for a JavaScript/TypeScript-centered full stack.	Frontend uses a configurable API base URL. Keep the provider adapter behind server routes.
Alternate: FastAPI	Python API with equivalent concepts: request models, routers, dependency configuration, and exception handling.	Use a separate backend environment and dependency file; do not run both servers on the same port.

5. Module Descriptions

Module	Responsibilities
News discovery	Load story metadata from configured sources, map categories, retain original URLs and timestamps, and communicate source or freshness.
Simplification service	Prepare a bounded prompt, call the selected provider, parse and validate structured output, and report when a fallback was used.
Custom simplifier	Accept pasted text, enforce size and content constraints, and pass it through the shared simplification pipeline.
Term explainer	Look up curated definitions or use a configured explanation path; provide definitions appropriate to novice readers.
API and validation	Expose stable HTTP routes, validate schemas, set timeouts, normalize errors, and apply CORS and rate-limiting policy.
Frontend presentation	Provide responsive pages, category navigation, article details, forms, result sections, accessibility, and empty/loading/error states.
Configuration and observability	Load secrets from environment or a secret manager; log request IDs and service status without logging credentials or unnecessary user content.

6. Technology Stack

Layer	Technology	Role
Frontend	React 19	Component-based user interface.
Build and development	Vite	Local development server and production asset build.
Styling	Tailwind CSS	Utility-based responsive styling.
Primary backend	Node.js + Express	REST API, validation, integrations, and fallback handling.
Alternate backend	Python + FastAPI	Optional REST API implementation with equivalent route semantics.
AI providers	Configurable provider adapter	Calls one of the provider/model options configured by the operator; availability and model names may change.
News data	Configurable news/feed source	Supplies articles or metadata; licensing, limits, and freshness depend on provider.
Transport	HTTP in local development; HTTPS in deployment	JSON requests and responses.

Version note: the brief names React 19 and illustrative model examples including Groq Llama 3.3 70B and Google Gemini 3.8 Flash. Those names are retained only as design context. The actual installed versions, model identifiers, APIs, pricing, and access limits must be verified against the project lockfiles and provider documentation at implementation time.

7. Project Structure

The following is a suggested layout for a repository that supports the primary JavaScript mode and keeps the alternate Python backend separate. Existing code may use different names.

```
finnews-ai/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── styles/
│   ├── package.json
│   └── vite.config.js
├── server/
│   ├── src/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── providers/
│   │   ├── middleware/
│   │   └── app.js
│   └── package.json
├── backend-fastapi/       # optional alternate mode
│   ├── app/
│   └── requirements.txt
├── .env.example
├── README.md
└── docs/
```

8. API Endpoint Documentation

The routes below define a proposed, implementation-neutral API contract for the client. They are examples for project planning, not evidence that a supplied codebase already implements them. Confirm paths, fields, status codes, and provider behavior against the actual server before publishing an API reference. Both backend modes should use the same contract where practical.

Method and path	Purpose	Example request fields	Typical response
GET /api/health	Basic service readiness check	None	status; service name; optional dependency status
GET /api/news?category=markets	List curated stories, optionally filtered	Query: category, page, limit	items[] with title, summary, source, URL, publishedAt, category
GET /api/news/:id	Retrieve one story record	Path: id	Story metadata; 404 if not found
POST /api/simplify	Simplify a story or supplied text	text or articleId; optional audience	Structured explanation sections and provider/fallback metadata
POST /api/terms/explain	Explain a financial term	term; optional context	term; definition; example if available
GET /api/terms?q=yield	Search available term entries	Query: q	Matching terms and short definitions

8.1 Illustrative response shape

A successful simplification response can be normalized to a shape such as the following. Field names are illustrative and may be adapted to the implementation:

```
{
  "headline": "Plain-language headline",
  "tldr": "Short summary",
  "what_happened": ["Source-supported fact"],
  "why_it_matters": ["Potential implication with uncertainty"],
  "key_takeaways": ["Takeaway"],
  "jargon": [{"term": "Basis point", "definition": "One hundredth of a percentage point."}],
  "source": {"title": "...", "url": "..."},
  "meta": {"provider": "configured", "fallback": false}
}
```

8.2 Error conventions

Use consistent HTTP status codes and a stable error envelope, for example {"error":{"code":"...","message":"...","requestId":"..."}}. A 400 response indicates invalid input; 404 indicates an unknown resource; 429 indicates a rate limit; and 500/502/503 indicate server or upstream failure. Do not return stack traces, provider credentials, or raw upstream error payloads to the browser. The exact mapping should be documented with the implementation.

9. Environment Setup and Execution

9.1 Prerequisites

- Install a supported Node.js LTS release for the client and Express server. Confirm the version required by the project package files.
- For the alternate mode, install a supported Python 3 release and create a virtual environment.

- Install Git if cloning the project repository.
- Obtain credentials only for the news or model services that the chosen implementation actually uses.

9.2 Configuration

Create local environment files from the supplied example template, if present. Keep real secrets in ignored local files or a deployment secret manager; never commit them or place them in frontend-prefixed variables that are embedded in browser assets.

Variable	Use	Example / handling
PORT	Backend listening port	3000 or another free local port.
CLIENT_ORIGIN	Allowed frontend origin	http://localhost:5173 for local development.
NEWS_API_KEY	Optional news-provider credential	Set only when required by configured news integration.
AI_PROVIDER	Select provider adapter	Use an implementation-supported identifier.
AI_API_KEY	Server-only model-provider credential	Never expose to browser code.
AI_MODEL	Model identifier supported by provider	Check provider dashboard/docs; avoid hard-coding obsolete names.
VITE_API_BASE_URL	Client API origin	May be public configuration; never store secrets here.

9.3 Windows setup: primary Express mode

6. Open PowerShell in the project folder.
7. Install dependencies in each package folder using the package manager specified by the lockfile. For npm: cd client; npm install, then repeat in server.
8. Copy .env.example to the backend local environment file and set the required values. Do not put server secrets in the client environment.
9. Start the backend in one terminal with npm run dev (or the script defined in server/package.json).
10. Start the client in a second terminal with npm run dev (or the script defined in client/package.json).
11. Open the local URL printed by Vite, commonly http://localhost:5173, and confirm the client API base URL targets the backend.

9.4 General environment setup: primary Express mode

On macOS or Linux, the same package steps apply from a terminal: cd client && npm install, then cd ../server && npm install. Configure the server environment, start the API, then start Vite from the client folder. Use the package scripts defined by the repository rather than assuming command names where they differ.

9.5 Optional FastAPI mode

Use this mode only if the Python service exists or is being implemented. Create a virtual environment in backend-fastapi, activate it (PowerShell: .\.venv\Scripts\Activate.ps1; macOS/Linux: source .venv/bin/activate), install the pinned dependencies from requirements.txt, configure its environment, and launch the application using the entry point documented by that backend (commonly uvicorn package.module:app --reload). Set the client API base URL to the FastAPI server. Do not run both backend modes on the same port.

9.6 Production deployment outline

Build the frontend using the project build script, deploy static assets through a suitable web host, and run one backend mode as a managed service. Configure HTTPS, environment-specific origins, secrets, logging, health checks, and upstream timeouts. Review news-source terms and model-provider data handling before production use.

10. Security Considerations

- Keep model and news API keys exclusively on the server; exclude local environment files from version control and rotate exposed keys.
- Validate request body types, text length, query parameters, and identifiers. Apply per-user or per-IP rate limits and request timeouts.
- Restrict CORS to known client origins in production; avoid wildcard origins when credentials are involved.
- Use HTTPS in production and secure deployment secrets through a managed secret store.
- Treat article text and model output as untrusted input. Escape rendered content; do not render model-generated HTML as trusted markup.
- Avoid logging complete submitted articles, secrets, authorization headers, or sensitive user data. Use request IDs and redacted operational metadata.
- Use a provider adapter with bounded retries, backoff for transient errors, concurrency limits, and explicit behavior for 429 responses.
- Preserve source attribution, dates, figures, and uncertainty; show the original source link alongside summaries.
- Present educational explanations with a clear notice that they are not individualized financial advice.
- Review upstream news licensing, provider terms, privacy disclosures, and applicable institutional requirements before deployment.

11. Testing Approach

Testing should verify behavior at the API boundary and user-visible states, with external services mocked in automated tests. Live provider integration checks should be optional and use controlled credentials.

Test level	Examples
Unit	Validate request schemas, text limits, category mapping, response normalization, error mapping, and fallback selection.
API integration	Check route success and invalid input; mock provider timeouts, 429 responses, malformed output, and unavailable news service.
Frontend component	Check category navigation, loading/error/empty states, form validation, accessible labels, and rendering of all explanation sections.
End-to-end	Run client and backend together using deterministic fixtures; submit text and verify a complete result is displayed.
Security and resilience	Check secret handling, CORS configuration, output escaping, rate limits, timeouts, and absence of sensitive values in logs.
Manual acceptance	Compare a range of articles against source facts; verify numbers, names, dates, caveats, and source links.

Acceptance criteria should include: the client remains usable when an upstream service is down; failures are understandable and do not claim a generated result where none exists; facts can be traced to the source; and sensitive keys are absent from built frontend files and browser network payloads.

12. Limitations and Assumptions

- The brief does not include a repository, package manifests, route implementation, schema, or deployment configuration. Proposed structure, endpoint paths, commands, and response shapes therefore require reconciliation against actual code.
- “Live and curated” news depends on a configured provider or feed, its coverage, refresh frequency, licensing, and rate limits.
- AI summaries can omit or distort context, numbers, or uncertainty. Human review and source checking remain important.
- Model names, APIs, quotas, availability, and pricing change over time; example model names in the concept are not guarantees.
- Offline or rate-limited fallbacks may be less current or less capable and must be identified clearly in the interface.
- A financial term definition can simplify nuance; context-specific meanings may need more than one definition.
- The application is an educational information tool and does not replace professional financial, legal, or tax advice.

13. Future Enhancements

- Add article deduplication, source reliability metadata, publication-time freshness indicators, and transparent feed provenance.
- Provide multilingual explanations, adjustable reading levels, and accessible audio summaries.
- Add user-selected topics, saved stories, reading history controls, and privacy-conscious personalization.
- Use retrieval-grounded definitions and source citations within each generated explanation.
- Introduce evaluation datasets and regression checks for factual retention, numeric fidelity, readability, bias, and unsupported claims.
- Support caching, queueing, observability dashboards, and operational alerting as traffic grows.
- Add authentication and saved preferences only when needed, with a clear privacy and retention policy.
- Offer a deployment guide, container configuration, CI checks, and documented API schema such as OpenAPI.

14. Conclusion

FinNews AI is designed to reduce the effort needed to understand financial news while keeping the reader connected to the original reporting. Its core value is a consistent explanation that distinguishes facts, possible relevance, takeaways, and terminology. A React/Vite client with an Express backend is the primary documented implementation; FastAPI can serve as an alternate backend if it follows the same client-facing contract. Reliable delivery depends on secure server-side credentials, careful source attribution, visible fallback behavior, and disciplined verification of AI-generated claims.

References

The project brief supplied the product purpose, intended audiences, feature set, preferred client stack, backend alternatives, and example provider names. The following official resources are starting points for implementation details; confirm current versions and terms before use.

12. React documentation. https://react.dev/ (accessed 26 October 2026).
13. Vite documentation. https://vite.dev/guide/ (accessed 26 October 2026).
14. Tailwind CSS documentation. https://tailwindcss.com/docs (accessed 26 October 2026).
15. Express documentation. https://expressjs.com/ (accessed 26 October 2026).
16. FastAPI documentation. https://fastapi.tiangolo.com/ (accessed 26 October 2026).
17. Node.js documentation. https://nodejs.org/docs/ (accessed 26 October 2026).
18. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/ (security implementation reference).
19. Configured news and AI provider documentation and service terms. Consult the provider actually selected for current model IDs, request schemas, limits, privacy terms, and pricing.

Presented by Mukta Kolhe